# Lakehouse Platform for Lending, Insurance and Recharge

Design Document — Data Engineer Take-Home Exercise

Aman Bhardwaj

# 1. The problem, as I read it

Three businesses — lending, insurance and recharge — each grew their own OLTP systems, event streams, partner files, third-party APIs and hand-maintained spreadsheets. That is a normal way for a company to arrive here, and exactly how you end up with the stated symptom: nobody can say with confidence whether a given table is current or correct. That sentence is the brief; everything below is in service of making it false.

I read "current or correct" as two separable and separately hard questions. "Correct" is the transformation being right — same raw fact in, same number out, deterministically. "Current" is honesty around time: the platform knowing what it does not yet know, and saying so, rather than publishing a plausible number built on an incomplete input. A design that solves correctness but not currency produces wrong answers with total confidence, which is worse than the status quo — the status quo at least makes people suspicious. The three hard problems map onto both.

Constraints taken as given: ~10k events/sec at peak, ~500M rows/day from files, 50M customers, five years of history, money as integer paise. None of the hard problems is about volume — 10k events/sec is roughly 5 MB/s. They are trust problems that happen to occur at a size where you cannot fix them by hand.

# 2. Architecture: what I chose, what I rejected, what it costs

Six zones left to right — sources, ingestion and transport, bronze, silver, publish gate, gold and serving — over a platform-services band that every zone depends on (diagram page 1). The flow is the design: every guarantee the platform makes is a property of one zone boundary, not of the system as a whole.

| Layer | Chosen | Rejected, and why | What the choice costs |
|---|---|---|---|
| Table format | Apache Iceberg on S3/Parquet | Delta Lake — nicer Spark ergonomics, but weaker branch/tag semantics for write-audit-publish and gravity toward one vendor. Hudi — best of the three at incremental upserts, but a larger operational surface than a small team should own. | A thinner SQL surface: no REPLACE WHERE, fewer MERGE conveniences, branch and tag operations as DDL rather than an API. Section 6.3 pays this explicitly. |
| Catalog | Iceberg REST catalog (Polaris or Nessie) | Hive Metastore — the default, rejected deliberately: a Thrift service over a 2010-era relational schema, no multi-table transactions, | A newer, smaller-community component on the critical path of every read and write, run by the team. Section 8 expects it to break |

| | | | |
|---|---|---|---|
| | | no branching. AWS Glue — zero-ops, but pins the catalog to one cloud while the engines above it stay portable. | first. |
| Processing | Spark (batch and Structured Streaming) | Flink — the better streaming engine, rejected because nothing downstream needs sub-second freshness. A second engine costs two sets of transformation code, two deployment paths and two on-call skill sets, to buy latency nobody asked for. 10k events/sec is ~5 MB/s; volume is not the constraint. | A floor of ~30–60s end-to-end, and micro-batch commit pressure on the catalog. A real-time fraud use case would reopen the decision, not tune it. |
| Query / serving | Trino for analysts; a KV store (ScyllaDB) for applications | Serving applications from Trino — a point lookup behind a query planner is tens of milliseconds at best, with no SLA under concurrency. One API over one table for all four consumers would serve all four badly. | Two serving paths to keep consistent, and a documented lag. The KV store is a cache with a rebuild path, never the system of record. |
| Orchestration | Dagster, partition-aware assets | Airflow — more common, but its unit is a task at a time of day, and the actual scheduling condition here is "has the input arrived for business date D". Encoding data dependencies in cron is how the current mess got made. | A less familiar tool, and asset-based thinking has a learning curve for engineers who arrive fluent in Airflow. |

Layer roles, briefly. Bronze is source-shaped, append-only and immutable, and never gets smarter transformations: its only job is to be a faithful replayable copy of what arrived and when, so that wrong silver logic six months from now is a recompute rather than an apology. Silver is conformed — CDC becomes ordered upserts, dimensions pick up SCD2 history, facts get a business_date from the feed's own settlement cutover, customers resolve to one identity across

the three businesses, and every monetary column is a BIGINT of paise. Gold is business-shaped marts, and nothing reaches it except through the publish gate.

## 3. Getting data in

Each source family gets the mechanism suited to how it actually behaves, and — more importantly — an explicit statement of what happens when it misbehaves.

| Source | How it lands | When it misbehaves |
|---|---|---|
| OLTP cores (lending, insurance, recharge) | Debezium → Kafka, keyed by primary key so per-key ordering survives partition assignment; applied to silver by the idempotent applier (6.1). | Connector restart or rebalance replays offsets: the LSN guard makes it inert. Schema change: the registry rejects an incompatible payload and the topic parks rather than writing garbage. Long outage: bronze retains the raw envelope, so catch-up is a replay, not a re-snapshot. |
| Event streams (~10k/sec peak) | Already event-shaped, so straight onto Kafka with a producer-assigned event_id; bronze append is a plain streaming write. Silver dedupes on event_id over a 24-hour window and buckets late events by event_time, not arrival time. | Producers retry, so duplicates are assumed rather than hoped against — the dedupe window is the design, and events later than 24 hours land in a late-arrival partition that triggers a targeted recompute of the affected business_date instead of being dropped or silently mixed in. |
| Partner / vendor files (~500M rows/day) | Claimed atomically in the ingest ledger on (source, business_date, file_hash) before a row is read; content hash, never filename, decides new vs duplicate vs correction (6.3). | Redelivery under a new name is a no-op. New content for a known date is a restatement, not an append. Non-arrival is what the publish gate is for. Malformed rows go to a per-source quarantine table with the reject reason, and a quarantine rate above a threshold is itself a P0 check. |
| Third-party APIs | Poller with an explicitly persisted cursor, so a restart resumes rather than re-paging from page 1. | Rate limiting and 5xx get bounded exponential backoff; the cursor does not advance past an unconfirmed page. A vendor silently changing pagination is the failure I cannot detect cheaply, so the ledger records row counts per poll and a discontinuity raises. |

| | | |
|---|---|---|
| Ops spreadsheets | Deliberately no magic pipeline. A governed CSV path: git-tracked, reviewed by PR, schema-validated in CI, then loaded as a reference table. | The real defect in a hand-maintained spreadsheet is not the file format, it is the absence of review before a number reaches production. Automating the load without adding the review would just automate the mistake. |

The three businesses are not interchangeable. Lending and payments settle at 23:00 IST, so business_date comes from that cutover, not calendar midnight; conflating them shifts a day of transactions. Insurance is the slowest and most restatement-heavy — claims reopen and reserves are revised months later — so insurance marts are the heaviest users of section 6.3 and need a much longer correction tail. Recharge is the opposite: high volume, low value per row, near-real-time expectations, negligible restatement, and the one family where I would accept a shorter completeness hold, because a delayed recharge dashboard costs more than an approximate one. I do not have the cutover times for insurance or recharge and did not invent them.

## 4. Physical design, capacity and cost

Several claims elsewhere in this document depend on the physical layout, so it is stated rather than assumed.

- **Partitioning —** facts and events partition by days(business_date); five years is ~1,825 partitions per table, which keeps manifest counts sane while letting the gate and the restatement overwrite address exactly one day. Very large facts add bucket(64, customer_id) as a second field so joins on customer are shuffle-light. CDC current-state tables in silver are bucketed on the primary key and not partitioned by date at all — rows would migrate partitions on update and rewrite files for no benefit.
- **Clustering —** sort order on (customer_id, event_time) within a partition, target file size 512MB, compaction to that size within an hour of a partition being published. Small files are the tax that micro-batching charges, and 1,440 commits/table/day generates them relentlessly.
- **Storage —** 500M rows/day at ~250 bytes/row compresses to ~25–40GB/day in Parquet: ~12–15TB/year, ~60–75TB over five years, plus a single-digit percentage for tag-pinned restatements. At S3 standard rates that is a few thousand dollars a month — not the interesting number.
- **Compute —** the interesting number. Streaming needs a standing cluster sized for peak, not average; file loads and mart rebuilds burst and can take spot capacity; compaction is a third standing draw that people forget to budget and that competes with ingestion for the same pool. I would instrument cost per table from day one — the total is not alarming, but the distribution is always more skewed than anyone expects, and that is where the savings are.

## 5. The unifying idea: five clocks

What made three problems feel like one design: every row here carries five distinct notions of time, and almost every subtle bug in a system like this comes from collapsing two of them. event_time — when the fact occurred. ingest_time — when the platform first saw it. business_date — which operational day it belongs to for settlement, not the calendar day it arrived. valid_time — the window during which a stated fact was considered true. knowledge_time — what the platform believed as of a given moment, which is the axis a restatement moves along and the axis an auditor's question is actually phrased in.

The realisation that made this tractable rather than a bitemporal-database research project: an Iceberg table's snapshot history already is a knowledge-time axis. Every commit records what the platform believed at that instant, for free, as a side effect of normal writes. I do not need to build bitemporal versioning — I need to stop discarding it and name the moments that matter. That is what 6.3 does, and it is why leakage-free training data for data scientists falls out of the same mechanism instead of needing a second one.

## 6. Three hard problems

I chose these over other candidates — schema evolution across three OLTP sources, cross-source identity resolution, spreadsheet governance — because they sit on the critical path and because getting any one of them wrong produces a failure that is invisible in production: the table stays queryable, nothing pages, and the wrongness surfaces months later when someone reconciles against a source. That is a much worse failure shape than a job that crashes, and it is the shape the brief's own pain statement describes. Code in dataplatform/; the reasoning that belongs next to a line of code lives in its docstring rather than being repeated here.

### 6.1 Applying CDC to Iceberg safely under replay

Why it's hard: the obvious implementation reads a micro-batch, MERGEs it, then commits offsets — two commits against two systems with no transaction spanning them. If the process dies between them the batch is re-applied, and whether that is harmless depends entirely on whether the MERGE is idempotent. A "last row wins" MERGE is not, because the replay can interleave differently with a later batch that has since landed for the same key.

Solution, and where the guarantee actually lives. Ordering is carried by the source database's own log position as _lsn — not source.ts_ms, which is not monotonic across a failover and can tie within one transaction — and the MERGE gates its UPDATE on s._lsn > t._lsn. That single predicate is both the ordering and the idempotency guarantee: replaying a batch whose rows are all at or below the target's current _lsn matches no UPDATE and inserts nothing, whether the replay is immediate or ten batches later. Deletes are soft — a versioned row with _deleted = true — so a late stale insert for a deleted key is rejected by the same comparison instead of resurrecting it, and every consumer reads a view that filters tombstones rather than being trusted to remember.

```
MERGE INTO target AS t USING staged AS s
ON t.pk = s.pk
WHEN MATCHED AND s._lsn > t._lsn THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

On top of that, a deterministic per-batch marker — a pure sha256 of the topic and sorted offset ranges — is appended to a small platform.cdc_applied_batches table after the data commit and probed before it. It is an optimisation, not a correctness mechanism: it skips a MERGE the LSN guard would have made inert anyway, and because it can only suppress work that was already provably no-op it cannot drift from the table in any way that matters, which is what makes a second table acceptable here at all. (Section 9 covers why I originally had this backwards.) Marker after the data, never before: a crash in between costs one redundant MERGE, where the reverse order would skip a batch that never landed and lose data.

Cost accepted: one partition-pruned lookup per micro-batch against a table of one small row per batch, on a 7-day retention. Note what it deliberately is not — a probe of the target on a _batch_id column. A sha256 is uniformly distributed, so per-file min/max bounds span almost the whole key space and Iceberg prunes nothing; that probe degrades to scanning a 50M-row table once a minute, and the column is mutable anyway because UPDATE SET * overwrites it. Rejected: offsets in the Iceberg snapshot summary via snapshot-property.*, atomic with the data commit but reachable only from DataFrameWriterV2, and MERGE is SQL.

## 6.2 Deciding a dataset is complete, and refusing to publish when it isn't

Why it's hard: completeness is not observable. You cannot know that no more data is coming for a business date, only whether everything you were promised has arrived. And the resulting failure is quiet in a specific way — if a mart joins a healthy loan feed against a stalled repayment feed, the output is queryable, non-empty, internally consistent and wrong: loans genuinely appear to have zero repayments, so no null check and no referential check fires. The only thing that catches it is refusing to publish past the slowest input.

Solution: completeness is declared per source in an arrival contract and decided against, never inferred from the data. Each upstream's watermark comes from the ingest ledger, a dataset's effective watermark is the minimum across its upstreams, and a missing watermark from any one collapses the result to unknown rather than being read optimistically. Publication is write-audit-publish on Iceberg branches: the candidate goes to a per-business-date branch no consumer reads, a graded suite of P0 (blocking) and P1 (warn-and-track) checks runs against the exact bytes on that branch, and only a clean run is fast-forwarded onto main in one atomic catalog call. A failure drops the branch and records HELD in an append-only manifest; main keeps serving the last good state.

There are two ways to get the watermark wrong, not one. A maximum across upstreams ignores the laggard — the obvious bug. A maximum within one upstream ignores holes: if a source reported D-3 and D but never D-2, max(business_date) says D and the gate waves through a month-end close built on a missing day. So source_watermark returns the newest date with an

unbroken run behind it, the only reading of "provably complete" that is actually provable. It assumes every calendar date is owed, so a source idle at weekends over-holds; the per-source calendar seam is where that gets fixed.

Cost accepted: a stalled upstream holds the whole dependent tree past its freshness SLA and pages someone. That is intended — for lending and settlement data a late number is recoverable and a wrong number that has been reported on is not — so the escape hatch is designed rather than stubbed. An Override carries an approver, a ticket and a reason, publishes the partition as OVERRIDDEN rather than PUBLISHED so consumers and auditors can tell the difference afterwards, and can waive the completeness hold but never a P0 check. Incompleteness is a judgement call; a negative money column is not.

### 6.3 Absorbing a correction without lying about what we used to believe

Why it's hard: the first two problems are about getting data in correctly; this one is about having been wrong in public. A settlement file for the 3rd arrives on the 7th with different totals, finance closed the month on the 5th using the old numbers, and an auditor later asks why the closed report and the live table disagree. Both answers have to remain available, and the difference has to be explainable.

Solution: every delivery is classified against the ingest ledger by content hash — NEW, DUPLICATE (byte-identical redelivery, a no-op) or RESTATEMENT (same date, different content). The lookup deliberately considers claimed-but-not-yet-complete rows as well as complete ones: restricting it to complete rows classifies a redelivery arriving while the first load is still in flight as brand new, and loads the same file twice. A restatement claims the next knowledge_version by MERGE into the ledger; Iceberg gives conflict detection, not automatic re-execution, so the retry loop is load-bearing rather than decoration, and the winner is established by reading its own worker_id back — "my MERGE did not raise" is not the same statement as "I own this claim".

The correction replaces the partition in one Iceberg commit via DataFrameWriterV2.overwrite(condition), detaching old files as it attaches new ones, so no reader sees the partition empty or doubled. It writes to the candidate branch, so a correction failing its own P0 checks is held like any other — a correction is not automatically an improvement. Invalidation fires from the publish path, not the write: telling marts to rebuild before the gate passes means the rebuild picks up old or new numbers depending on timing. On publish a tag is cut — DDL, ALTER TABLE … CREATE TAG … RETAIN 1825 DAYS — while ordinary snapshots expire after 7 days, so "as reported on 31 March" becomes VERSION AS OF 'close_2026_03_31': an ordinary SQL read an auditor runs themselves.

Cost accepted, and one I think is genuinely unresolved. Tags pin data files, so superseded partition versions stay on disk for five years — a single-digit percentage of storage, and a bargain against not being able to answer the auditor. The sharper cost: pinning files for five years is in direct tension with a data-subject erasure request under the DPDP Act. You cannot delete rows a tag is holding, and "we keep it for audit" is not obviously a lawful basis for retaining personal data

against an erasure request. My answer is to encrypt subject-identifying columns under a per-subject KMS key and destroy the key rather than the rows, so tagged historical reads return ciphertext — the audit trail keeps its shape and its totals without keeping the personal data. That costs a KMS on the read path for those columns, and carries a real risk that a regulator considers crypto-shredding insufficient. It is the constraint most likely to force a redesign here, and I would want legal in the room before committing to it.

## 7. Serving: what each consumer actually gets

- **Analysts and finance —** gold marts through Trino, views filtered on the manifest's valid_through, so a dashboard physically cannot render a partition the gate refused — the trust boundary sits in the serving path, not in a runbook people must remember.
- **Applications —** sub-millisecond lookups from a KV store fed off conformed silver. A cache with a documented lag and a rebuild path, never the system of record.
- **Data scientists —** read as of the Iceberg tag current at their training cutoff, making label leakage from a later correction structurally impossible rather than a code-review concern. Same mechanism as the auditor's — no second system.
- **Auditors —** time travel, lineage, and the manifest of every gate decision including refusals. "We refused this partition, for these reasons, at this time" is the evidence actually asked for six months later.

## 8. Living with it: run, fail, recover, and where it breaks first

Day to day: Dagster materialises partition-scoped assets when their inputs report complete, not at a time of day. Prometheus and Grafana watch four things per table — freshness against the arrival contract, completeness, reconciliation delta against source control totals, and cost. A held partition pages; a P1 raises a ticket. Compaction, snapshot expiry and orphan cleanup run as a standing maintenance job, which is what keeps 6.3's audit history affordable rather than an unbounded liability.

Recovery has three shapes. A failed job is a retry, and every path here is built to be re-runnable. Bad transformation logic is a bronze replay — recompute silver and gold for the affected dates, which is why bronze stays immutable and why five years of it earns its storage. Bad published data is the restatement path, the only one needing a human decision rather than an operator, because someone has to say whether finance re-closes. Target: any single business date recomputable within one business day — a claim about the shape of the pipeline, not a benchmarked number.

Where it breaks first: metadata accumulation in the catalog, not storage or compute. At ~200 tables under CDC on a 1-minute trigger the platform takes 1,440 snapshots per table per day — ~288,000 across the platform, which at 3.3 commits/sec in aggregate is not a throughput

problem, and I want to be careful not to claim it is. The concern is the accumulation: with 7-day expiry that is ~10,000 live snapshots per table, each referencing manifests, and commits to one table serialise against each other, so per-table latency grows with metadata size and expiry has to keep pace or it compounds. That is arithmetic, not a benchmark. Second: maintenance falling behind during a burst — a festival spiking recharge volume — because compaction competes with the ingestion it is cleaning up after, and the backlog degrades reads slowly enough that nobody notices until queries are already slow. Third, and the one I would watch soonest in practice: the gate's blast radius. One chronically-late upstream can hold a large dependent subtree, and that failure is not technical — it is that people start asking for overrides routinely, and a routine override is a disabled gate with extra paperwork.

# 9. Honesty section

## What I deliberately left out

- **Cross-business identity resolution —** designed the mechanism (per-source resolution across three business lines) but did not write the matching logic. It is its own hard problem and I judged the three I picked to be sharper and more central to the stated pain.
- **Insurance and recharge cutover times —** I state 23:00 IST as the business-date boundary for lending and payments because the given facts imply a lending/payments flow, but I do not have the operational cutover times for insurance or recharge and did not invent settlement-specific logic for them. The section 3 differentiation is about behaviour I can reason about — restatement tails, volume, latency tolerance — not about numbers I made up.
- **Schema evolution across three OLTP sources —** the registry rejects incompatible payloads and the topic parks, which is a policy, not a design. Additive change should flow through automatically and does not here.
- **Alerting and approval plumbing —** alerting integration and the workflow that mints an Override are named seams (_page, _raise_warning, the approval path). The Override itself is now designed and tested; who is allowed to sign one is an organisational decision I do not have the context to make.
- **The erasure-versus-audit resolution —** the crypto-shredding answer in 6.3 is a direction, not a design. I have not worked out key-per-subject at 50M customers, the KMS request rate on the read path, or what happens to a tagged read whose key is gone mid-query.

## Where I'm not fully sure

- **The 6-hour due_after_close default —** confident in the shape — declare it, don't infer it — but I have not load-tested whether 6 hours is realistic for every dataset at this volume, and it is almost certainly too generous for recharge and too tight for insurance.

- **Whether the catalog is really the first bottleneck —** back-of-envelope arithmetic, not a benchmark against a real REST catalog under this load. It is the number I would most want to measure before committing to a 1-minute trigger across 200 tables.
- **The contiguity rule —** the hole-detection watermark assumes every calendar date is owed. For a genuinely intermittent source that is wrong and over-holds, and I have not designed the per-source calendar that fixes it.
- **Overwrite atomicity under concurrent readers —** 6.3 asserts that an overwrite commit is never observed half-applied. That is documented Iceberg behaviour and it is also the property I would be least surprised to find broken under real concurrent load, and the one I have not tested.
- **The entry points, and the suite itself —** run_stream, ingest_delivery and publish_restatement are the three top-level entry points and none is exercised by a test — the guarantees underneath each are, the wiring is not. And I have not run the suite: it is written against a real catalog rather than mocks, but unexecuted, which is the first gap I would close.

## What changed on review, and where AI was used

Two things here are materially different from my first pass, and both matter more than anything I got right first time. First, I had claimed the per-batch marker was the idempotency guarantee. It is not: it is mutable, since UPDATE SET * overwrites it, and the probe I described as metadata-pruned would in fact have scanned the table, because a sha256 defeats min/max pruning. The LSN predicate was doing the real work all along, 6.1 now says so, and the tests are restructured to prove correctness with the marker deliberately removed. Second, source_watermark took a maximum within each upstream, so a hole in one source's history passed the gate — the exact class of failure the gate exists to prevent, hiding inside the function that implements it. Both came from reading my own code against my own claims rather than from a test, which is itself worth noting.

AI assistance: used to draft and iterate the code, tests and this document against a design I had already decided on, to generate the draw.io XML from a layout I specified box-by-box, and as an adversarial reviewer of my first pass — which is where the two corrections above came from. Every API it produced I checked against the Iceberg documentation rather than trusting, and that is what caught a REPLACE WHERE clause that reads naturally and is Delta syntax Iceberg does not have, plus system.create_tag, system.create_branch and system.drop_branch calls that do not exist — branches and tags are DDL. Exactly the errors a fluent-sounding draft produces and a reviewer who reads code as code will catch. The decisions are mine: the choice of these three problems, the min-not-max and no-holes-behind-it watermark rules, missing-as-unknown, the LSN predicate as the guarantee with the marker demoted to an optimisation, content-hash classification, tags-as-knowledge-time, the override-waives-completeness-never-a-P0 line, and the five-clocks framing that ties them together.

One deliberate deviation: the brief suggests roughly 200–400 lines of code. dataplatform/ is about 1,040 lines, of which 315 are logic once docstrings are stripped, plus ~820 lines of tests. So

the logic sits inside the suggested range and the overage is entirely reasoning kept next to the line it defends — which the brief also asks for, since the code is where a guarantee either holds or does not. I trimmed the module headers once the design document covered the same ground, and left the per-function reasoning. Flagging the number rather than hoping it goes unnoticed.